

Boletín de ejercicios iterativos

Transformación entre funcional <-> imperativo

- Ej-B1. Transforme a iterativo utilizando while la siguiente función que suma los elementos de una lista:

```
Double sumLista(List<Double> list) {  
    return IntStream.range(0, list.size()).mapToDouble(i -> list.get(i)).sum();  
}
```

- Ej-B2. Transforme a iterativo utilizando while la siguiente función:

```
Integer sumRange(Integer a, Integer b) {  
    return IntStream.range(a, b).sum();  
}
```

- Ej-B3. Transforme a iterativo utilizando while la siguiente función:

```
Double sumIterate(Double a, Double b, Double c) {  
    Double e0 = a;  
    Predicate<Double> g = e -> e < b;  
    UnaryOperator<Double> nx = e -> e * c;  
    return Stream.iterate(e0, g, nx).mapToDouble(Double::doubleValue).sum();  
}
```

- Ej-B4. Transforme a iterativo utilizando while la siguiente función:

```
Double sumaCuadrados(Integer a, Integer b) {  
    return IntStream.range(a, b).mapToDouble(e -> e * e).sum();  
}
```

- Ej-B5. Transforme a iterativo utilizando while la siguiente función:

```
Integer sumaMultiplos(Integer a, Integer b, Integer m) {  
    return IntStream.range(a, b).filter(e -> e % m == 0).sum();  
}
```

- Ej-B6. Transforme a iterativo utilizando while la siguiente función:

```
Integer sumaElems(List<Set<Integer>> ls) {  
    return ls.stream().flatMap(e -> e.stream()).mapToInt(Integer::intValue).sum();  
}
```

- Ej-B7. Transforme a iterativo utilizando while la siguiente función que comprueba si un número es primo:

```
Boolean esPrimo(Long n) {  
    if (n == 2)  
        return true;  
    if (n <= 1 || n % 2 == 0)  
        return false;  
    Long sqrt = (long) Math.sqrt(n);  
    return LongStream.iterate(3, i -> i <= sqrt, i -> i + 2).noneMatch(x -> n % x == 0);  
}
```

- Ej-B8. Transforme a iterativo utilizando while la siguiente función que devuelve el siguiente número primo mayor que n :

```

Long siguientePrimo(Long n) {
    if (n < 2)
        return 2L;
    Long e0 = (n % 2 == 0) ? n + 1 : n + 2;
    return Stream.iterate(e0, e -> e + 2).filter(e -> esPrimo(e)).findFirst().get();
}

```

- Ej-B9. Transforme a iterativo utilizando while la siguiente función que suma los n primeros números primos:

```

Long sumaPrimerosPrimos(Integer n) {
    if (n < 1)
        return 0L;
    return Stream.iterate(2L, e -> siguientePrimo(e)).limit(n).mapToLong(Long::longValue).sum();
}

```

- Ej-B10. Transforme a iterativo utilizando while la siguiente función que comprueba si una lista está ordenada ascendentemente:

```

<E> Boolean esOrdenada(List<E> ls, Comparator<E> cmp) {
    return IntStream.range(0, ls.size() - 1).allMatch(i -> cmp.compare(ls.get(i), ls.get(i + 1)) <= 0);
}

```

- Ej-B11. Transforme a iterativo utilizando while la siguiente función que devuelva una lista de pares de un primo y su siguiente, tal que el primero sea mayor o igual que m y menor que n , y que la diferencia entre ellos sea una cantidad k (siendo $k \geq 2$).

```

// Para usar clase Pair incluir import us.lsi.common.Pair; (proyecto ParteComun)
// y para Stream2 incluir import us.lsi.streams.Stream2; (proyecto ParteComun)
List<Pair<Long, Long>> primosPar(Long m, Long n, Integer k) {
    Stream<Long> sPrimos = Stream.iterate(siguientePrimo(m - 1), x -> x < n, x -> siguientePrimo(x));
    Stream<Pair<Long, Long>> sTuplas = Stream2.consecutivePairs(sPrimos);
    return sTuplas.filter(t -> (t.second() - t.first()) == k).collect(Collectors.toList());
}

```

- Ej-B12. Transforme a iterativo utilizando while la siguiente función que devuelva el número de veces que se repite cada divisor primo encontrado para todos los números del intervalo $[m, n]$.

```

Map<Long, Long> repDivisores(Long m, Long n) {
    return Stream.iterate(m, x -> x < n, x -> x + 1).flatMap(x -> divisores(x))
        .collect(Collectors.groupingBy(Long::longValue, Collectors.counting()));
}
Stream<Long> divisores(Long n) {
    return Stream.iterate(2L, x -> x <= n, x -> siguientePrimo(x)).filter(x -> n % x == 0);
}

```

- Ej-B13. Transforme a iterativo utilizando while la siguiente función:

```

Long iterativoFuncional1(List<Long> ls, Integer max_elems) {
    return ls.stream().map(e -> siguientePrimo(e)).filter(e -> e > 10).limit(max_elems)
        .collect(Collectors.summingLong(e -> e));
}

```

- Ej-B14. Transforme a iterativo utilizando while la siguiente función:

```

Long iterativoFuncional2(List<Long> ls, Integer max_elems) {
    return ls.stream().limit(max_elems).map(e -> siguientePrimo(e)).filter(e -> e > 10)
        .collect(Collectors.summingLong(e -> e));
}

```

- Ej-B15. Transforme a iterativo utilizando while la siguiente función:

```

Long iterativoFuncional3(List<Long> ls, Integer max_elems) {
    return ls.stream().filter(e -> e > 10).limit(max_elems).map(e -> siguientePrimo(e))
        .collect(Collectors.summingLong(e -> e));
}

```

- Ej-B16. Transforme a iterativo utilizando while la siguiente función:

```

// Para IntPair incluir import us.lsi.common.IntPair; (proyecto ParteComun)
Integer mcd(Integer a, Integer b) {
    return Stream.iterate(IntPair.of(a, b), t -> IntPair.of(t.second(), t.first() % t.second()))
        .dropWhile(t -> !t.second().equals(0)).findFirst().get().first();
}

```

- Ej-B17. Transforme a iterativo funcional la siguiente función:

```

Integer suma(List<Integer> list) {
    Iterator<Integer> it = list.iterator();
    Integer a = 0;
    while (it.hasNext()) {
        Integer e = it.next();
        a = a + e;
    }
    return a;
}

```

- Ej-B18. Transforme a iterativo funcional la siguiente función:

```

String fWhile(Integer a, Integer b) {
    String ac = "";
    while (!(a < 5 || b < 5)) {
        ac = ac + String.format("%d", a + b);
        a = a / 2;
        b = b - 2;
    }
    return ac + String.format("(%d)", a * b);
}

```

- Ej-B19. Transforme a iterativo funcional la siguiente función:

```

Long gIt(Long n) {
    Long a = 0L;
    Long b = 1L;
    Long c = 2L;
    Integer i = 0;
    while (i < n) {
        Long a0 = a;
        a = b;
        b = c;
        c = 2 * c + 3 * a0 + 5;
        i = i + 1;
    }
    return a;
}

```

- Ej-B20. Transforme a iterativo funcional la siguiente función:

```
Long fIt(Integer n) {  
    List<Long> f = new ArrayList<Long>();  
    f.add(0L);  
    f.add(1L);  
    Integer i = 2;  
    while (i <= n) {  
        f.add(i, f.get(i-1) + f.get(i-2));  
        i = i + 1;  
    }  
    return f.get(n);  
}
```

Transformacion entre iterativo <-> recursivo final

- Ej-C1. Transforme a recursivo final:

```
Double sumLista(List<Double> list) {
    Integer i = 0;
    Double b = 0.;
    while (i < list.size()) {
        b = b + list.get(i);
        i = i + 1;
    }
    return b;
}
```

- Ej-C2. Transforme a recursivo final:

```
Boolean esPrimo(Long n) {
    if (n == 2)
        return true;
    if (n <= 1 || n % 2 == 0)
        return false;
    Long sqrt = (long) Math.sqrt(n);
    Long i = 3L;
    Boolean b = false;
    while (i <= sqrt && !b) {
        b = n % i == 0;
        i = i + 2;
    }
    return !b;
}
```

- Ej-C3. Transforme a recursivo final:

```
Long siguientePrimo(Long n) {
    if (n < 2)
        return 2L;
    Long e = (n % 2 == 0) ? n + 1 : n + 2;
    Long r = null;
    while (r == null) {
        if (esPrimo(e)) {
            r = e;
        }
        e = e + 2;
    }
    return r;
}
```

- Ej-C4. Transforme a recursivo final:

```
Long sumaPrimerosPrimos(Integer n) {
    if (n < 1)
        return 0L;
    Long ac = 0L;
    Integer i = 0;
    Long p = 2L;
    while (i < n) {
        ac = ac + p;
        p = siguientePrimo(p);
        i = i + 1;
    }
    return ac;
}
```

- Ej-C5. Transforme a recursivo final:

```

<E> Boolean esOrdenada(List<E> ls, Comparator<E> cmp) {
    Integer i = 0;
    Boolean a = true;
    while (i < ls.size() - 1 && a) {
        a = cmp.compare(ls.get(i), ls.get(i + 1)) <= 0;
        i++;
    }
    return a;
}

```

- Ej-C6. Transforme a recursivo final:

```

Integer mcd(Integer a, Integer b) {
    while (b != 0) {
        Integer a0 = a;
        a = b;
        b = a0 % b;
    }
    return a;
}

```

- Ej-C7. Transforme a recursivo final:

```

Long iterativoImperativo(List<Long> ls, Integer max_elems) {
    Long ac = 0L;
    Integer i = 0, contador = 0;
    while (i < ls.size() && contador < max_elems) {
        Long t = siguientePrimo(ls.get(i));
        if (t > 10) {
            ac = ac + t;
            contador = contador + 1;
        }
        i = i + 1;
    }
    return ac;
}

```

- Ej-C8. Transforme a recursivo final:

```

// Para usar clase Pair incluir import us.lsi.common.Pair; (proyecto ParteComun)
List<Pair<Long, Long>> primosPar(Long m, Long n, Integer k) {
    Long e = siguientePrimo(m - 1);
    Long ant = null;
    List<Pair<Long, Long>> ac = new ArrayList<Pair<Long, Long>>();
    while (e < n) {
        if (ant != null && (e - ant) == k) {
            Pair<Long, Long> t = Pair.of(ant, e);
            ac.add(t);
        }
        ant = e;
        e = siguientePrimo(e);
    }
    return ac;
}

```

- Ej-C9. Transforme a recursivo final:

```

Map<Long, Long> repDivisores(Long m, Long n) {
    Long i = m;
    Map<Long, Long> ac = new HashMap<Long, Long>();
    while (i < n) {
        List<Long> lis = divisores(i);
        int j = 0;
        while (j < lis.size()) {
            Long e = lis.get(j);
            if (!ac.containsKey(e)) {
                ac.put(e, 1L);
            } else {
                ac.put(e, ac.get(e) + 1);
            }
            j = j + 1;
        }
        i = i + 1;
    }
    return ac;
}

List<Long> divisores(Long n) {
    List<Long> ac = new ArrayList<Long>();
    Long s = 2L;
    while (s <= n) {
        if (n % s == 0)
            ac.add(s);
        s = siguientePrimo(s);
    }
    return ac;
}

```

Diseño funcional (utilizando la API Streams de Java)

- Ej-D1. Implemente la función *esPrimo* que reciba un parámetro *n* de tipo Long y devuelva si la entrada es un número primo. Ayuda: optimice la búsqueda hasta la raíz cuadrada de la entrada.
- Ej-D2. Implemente la función *siguientePrimo* que reciba un parámetro *n* de tipo Long y devuelva el primer número primo mayor que *n*. Ayuda: reutilice la función anterior *esPrimo*, y optimice la búsqueda saltándose los números pares. Filtre y quedese con el primer elemento que sea mayor que *n*.
- Ej-D3. Implemente la función *sumaPrimerosPrimos* que reciba un parámetro *n* de tipo Integer y devuelva la suma (long) de los primeros *n* números primos. Reutilice la función anterior *siguientePrimo*.
- Ej-D4. Implemente la función *primosIntervalo* que reciba los parámetros *m* y *n* y devuelva la suma de los números primos mayores o iguales que *m* y menores que *n*. Reutilice la función anterior *siguientePrimo*.
- Ej-D5. Implemente la función *List hastaMayorN(List lista, Integer n)* que devuelva los elementos de la *lista* que van desde el inicio hasta el primer elemento que sea mayor estricto que *n* (sin incluir dicho elemento). Ayuda: utilice el método *takeWhile* de la clase *Stream*.
- Ej-D6. Implemente la función *List desdeMayorN(List lista, Integer n)* que devuelva los elementos de la *lista* que van desde primer elemento que sea mayor estricto que *n* (incluir dicho elemento) hasta el final de la lista. Ayuda: utilice el método *dropWhile* de la clase *Stream*.
- Ej-D7. Implemente la función *Boolean esOrdenada(List ls, Comparator cmp)* que devuelva un booleano indicando si la lista está ordenada de forma ascendente.
- Ej-D8. Implemente la función *List<Pair<Long,Long>> primosPar(Long m, Long n, Integer k)* que devuelva una lista de pares de un primo y su siguiente, tal que el primero sea mayor o igual que *m* y menor que *n*, y que la diferencia entre ellos sea una cantidad *k* (siendo $k \geq 2$).
- Ej-D9. Implemente la función *Map<Long, Long> repDivisores(Long m, Long n)* que devuelva el número de veces que se repite cada divisor primo encontrado para todos los números del intervalo [*m*, *n*).
- Ej-D10. Implemente la función *Boolean esAritmetica(List ls)* que devuelva verdadero si la lista de enteros forma una serie aritmética, y falso en caso contrario.